Karolina Antonowicz, PB117296, grupa A

In [6]:
```
import pandas as pd
```

In [77]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE
```

In [52]:
```
import matplotlib.pyplot as plt
%matplotlib inline
```

Wykorzystany w zadaniu zbiór danych Iris zawiera 150 obserwacji trzech gatunków kwiatów: Setosa, Versicolor oraz Virginica. Każdy z gatunków reprezentowany jest przez 50 próbek, co czyni zbiór idealnie zbalansowanym. Dane opisane są czterema cechami numerycznymi: długością i szerokością płatka oraz działki kielicha

In [18]:
```
# Wczytanie zbioru danych
iris=load_iris()
```

In [20]:
```
iris.feature_names
```

Out[20]:
```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

In [21]:
```
iris.target_names
```

Out[21]:
```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

Przed przystąpieniem do modelowania przeprowadzono analizę struktury danych. Sprawdzono statystyki opisowe oraz brakujące wartości, aby zapewnić spójność zbioru.

In [37]:
```
# Tworzenie obiektu DataFrame dla lepszej czytelności danych
df = pd.DataFrame(iris.data,columns=iris.feature_names)
df
```

Out[37]:

| | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) |
|---|---|---|---|---|
| **0** | 5.1 | 3.5 | 1.4 | 0.2 |
| **1** | 4.9 | 3.0 | 1.4 | 0.2 |
| **2** | 4.7 | 3.2 | 1.3 | 0.2 |
| **3** | 4.6 | 3.1 | 1.5 | 0.2 |
| **4** | 5.0 | 3.6 | 1.4 | 0.2 |
| **...** | ... | ... | ... | ... |
| **145** | 6.7 | 3.0 | 5.2 | 2.3 |
| **146** | 6.3 | 2.5 | 5.0 | 1.9 |
| **147** | 6.5 | 3.0 | 5.2 | 2.0 |
| **148** | 6.2 | 3.4 | 5.4 | 2.3 |
| **149** | 5.9 | 3.0 | 5.1 | 1.8 |

150 rows × 4 columns

In [41]:
```
df['species'] = iris.target
```

Statystyki opisowe

In [42]:
```
df.describe()
```

Out[42]:

| | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) | species |
|---|---|---|---|---|---|
| **count** | 150.000000 | 150.000000 | 150.000000 | 150.000000 | 150.000000 |
| **mean** | 5.843333 | 3.057333 | 3.758000 | 1.199333 | 1.000000 |
| **std** | 0.828066 | 0.435866 | 1.765298 | 0.762238 | 0.819232 |
| **min** | 4.300000 | 2.000000 | 1.000000 | 0.100000 | 0.000000 |
| **25%** | 5.100000 | 2.800000 | 1.600000 | 0.300000 | 0.000000 |
| **50%** | 5.800000 | 3.000000 | 4.350000 | 1.300000 | 1.000000 |
| **75%** | 6.400000 | 3.300000 | 5.100000 | 1.800000 | 2.000000 |
| **max** | 7.900000 | 4.400000 | 6.900000 | 2.500000 | 2.000000 |

Sprawdzenie liczebności klas

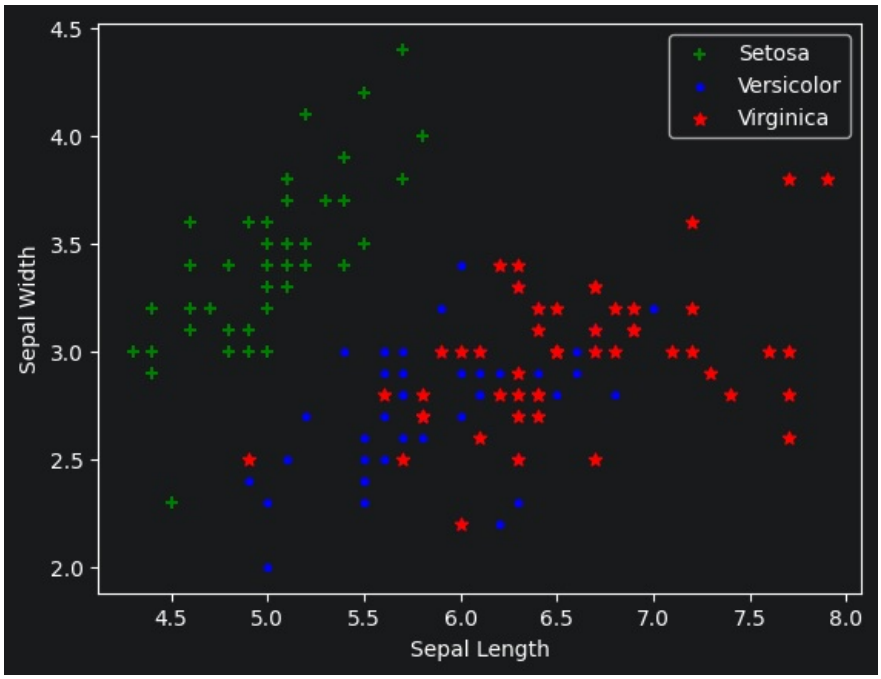
In [43]:
```
df['species'].value_counts()
```

Out[43]:
```
species
0    50
1    50
2    50
Name: count, dtype: int64
```

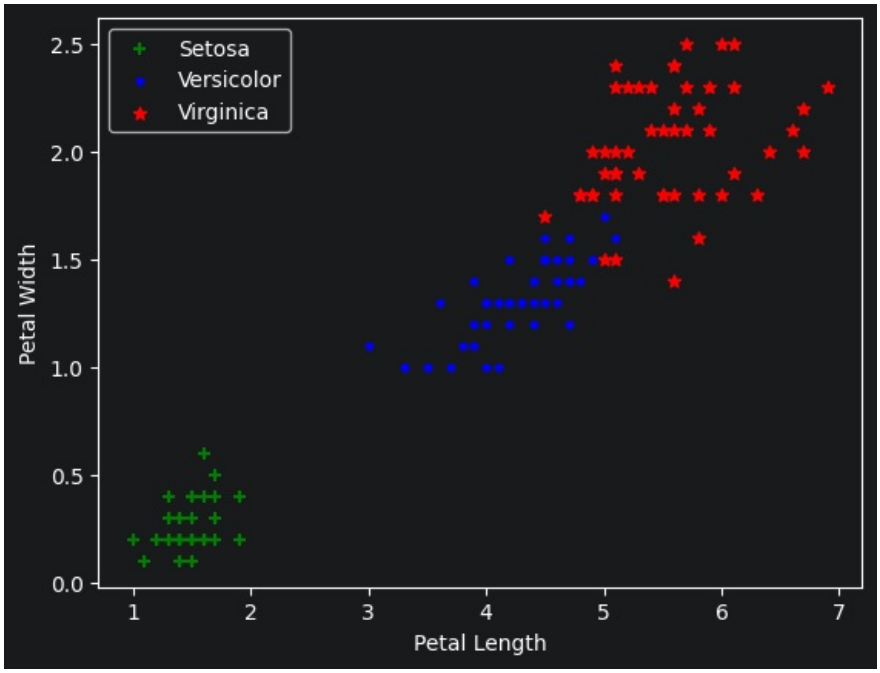
Podział zbioru danych na mniejsze części

In [200…
```
df0 = df[:50] # setosa
df1 = df[50:100] # versicolor
df2 = df[100:] # virginica
```

In [194…
```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.scatter(df0['sepal length (cm)'], df0['sepal width (cm)'], color='green', marker='+', label='Setosa')
plt.scatter(df1['sepal length (cm)'], df1['sepal width (cm)'], color='blue', marker='.', label='Versicolor')
plt.scatter(df2['sepal length (cm)'], df2['sepal width (cm)'], color='red', marker='*', label='Virginica')
plt.legend()
plt.show()
```




In [195…
```
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.scatter(df0['petal length (cm)'], df0['petal width (cm)'], color='green', marker='+', label='Setosa')
plt.scatter(df1['petal length (cm)'], df1['petal width (cm)'], color='blue', marker='.', label='Versicolor')
plt.scatter(df2['petal length (cm)'], df2['petal width (cm)'], color='red', marker='*', label='Virginica')
plt.legend()
plt.show()
```




Wizualizacja pozwala zaobserwować, że gatunek Setosa jest wyraźnie odseparowany od pozostałych, natomiast Versicolor i Virginica częściowo na siebie nachodzą.

In [95]:
```
X, y = iris.data, iris.target
```

In [96]:
```
# Standaryzacja cech
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Dane zostały podzielone na zbiór treningowy (80%) i testowy (20%). Ponieważ algorytm KNN bazuje na obliczaniu odległości euklidesowej, zastosowano skalowanie cech (StandardScaler). Zapewnia to, że żadna z cech nie zdominuje modelu tylko ze względu na większą skalę liczbową.

In [102…
```
# Podział na zbiór treningowy i testowy
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

Do klasyfikacji wykorzystano model KNN z parametrem k=9. Wybór liczby sąsiadów jest kluczowy dla uniknięcia przeuczenia (overfittingu) przy zachowaniu wysokiej precyzji.

In [196…
```
# Inicjalizacja i trenowanie modelu
knn = KNeighborsClassifier(n_neighbors=9)
knn.fit(X_train, y_train)
```

Out[196…
```
▾ KNeighborsClassifier
▶ Parameters
```

In [198…
```
# Wynik dopasowania
accuracy = knn.score(X_test, y_test)
print(f"Ogólna dokładność modelu: {accuracy}")
```

```
Ogólna dokładność modelu (Accuracy): 0.9666666666666667
```

In [167…
```
y_pred = knn.predict(X_test)
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        11
  versicolor       1.00      0.92      0.96        13
   virginica       0.86      1.00      0.92         6

    accuracy                           0.97        30
   macro avg       0.95      0.97      0.96        30
weighted avg       0.97      0.97      0.97        30
```

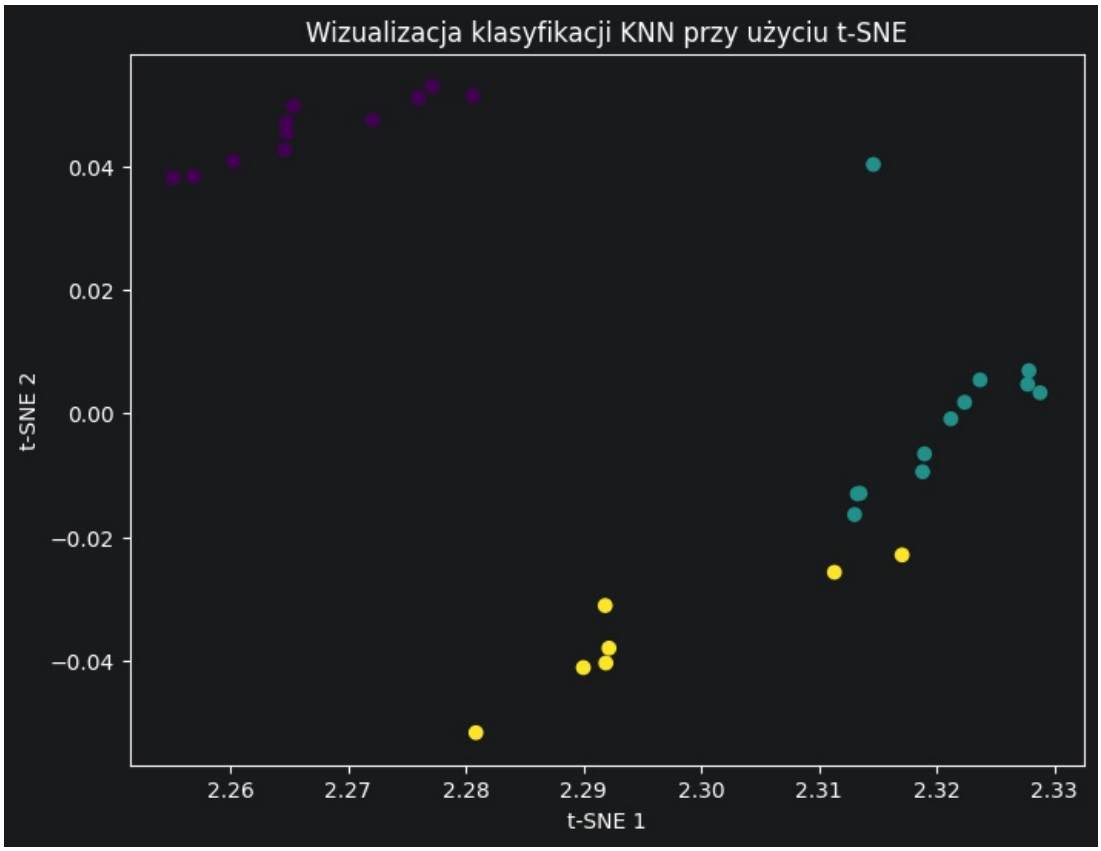
Uzyskane wyniki (Accuracy ok. 97%) świadczą o bardzo wysokiej skuteczności modelu.

- Precision: Pokazuje, jak wiele z próbek wskazanych jako dany gatunek faktycznie do niego należy.\
- Recall: Mówi o tym, ile próbek z danego gatunku udało się poprawnie wykryć. Najmniejszą precyzję odnotowano dla gatunku Virginica, co wynika z naturalnego podobieństwa niektórych jej cech do gatunku Versicolor.

- F1-Score: Jest średnią harmoniczną precyzji i czułości. Jest to najbardziej miarodajny wskaźnik, gdy zależy nam na zachowaniu balansu między wykrywaniem wszystkich próbek a unikaniem błędnych przypisań. Wynik bliski 1.00 świadczy o niemal idealnym dopasowaniu modelu.
- Support: Określa liczebność próbek każdego gatunku w zbiorze testowym (np. 11 dla Setosa). Pozwala to ocenić statystyczną wiarygodność obliczonych metryk.

In [175…
```
tsne = TSNE(n_components=2,perplexity=29, random_state=10)
X_tsne = tsne.fit_transform(X_test)
```

In [176…
```
plt.figure(figsize=(8, 6))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_pred, cmap='viridis')
plt.title("Wizualizacja klasyfikacji KNN przy użyciu t-SNE")
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.show()
```




Ponieważ dane są 4-wymiarowe, zastosowano algorytm t-SNE w celu redukcji wymiarowości do 2D. Wykres wizualizuje wyniki klasyfikacji na płaszczyźnie. Wyraźne skupiska punktów potwierdzają, że algorytm KNN poprawnie zidentyfikował strukturę grup w danych.

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js